

P2544R0

C++ exceptions are becoming more and more problematic

Published Proposal, 2022-02-12

Author:

[Thomas Neumann](#) (TUM)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

URL:

<https://wg21.link/P2544/0>

Source:

<https://github.com/neumannt/exceptionperformance/blob/master/paper/exceptionperformance.bs>

Date:

2022-02-07

Abstract

Current hardware trends make C++ exceptions harder and harder to justify. This paper illustrates and quantifies the problem and discusses potential future directions to fix exceptions.

Table of Contents

- 1 Introduction
- 2 Root cause
- 3 Alternatives
 - 3.1 `std::expected`

- 3.2 boost::LEAF
- 3.3 throwing values
- 3.4 fixing traditional exceptions

4 Moving Forward

5 Acknowledgments

References

Informative References

§ 1. Introduction

Many projects avoid or even actively disable C++ exceptions due to a number of reasons (see [\[P0709R4\]](#) for a detailed discussion). The unfortunate reality is that, while exceptions are the default error reporting mechanism in C++, there are good reasons for avoiding them. In fact the current trend to high core counts makes exceptions unsustainable, at least in their current implementation. In the following, we first illustrate and quantify the problem, and then discuss potential mitigations. The source code for all experiments is available at [\[ep\]](#).

As illustrational example consider this small code fragment:

```
struct invalid_value {};  
  
void do_sqrt(std::span<double> values) {  
    for (auto& v : values) {  
        if (v < 0) throw invalid_value{};  
        v = std::sqrt(v);  
    }  
}
```

It performs a somewhat expensive computation and throws an exception if an invalid value is encountered. Its performance depends on the likelihood of encountering an exception. To test the performance, we call it 100'000 times with an array of 100 doubles with the value 1.0. With a certain probability, we set one value of that array to -1 to trigger an error. On an AMD Ryzen 9 5900X we observe the following execution numbers (in milliseconds) for the whole workload, depending on the thread count and the failure rate:

Threads	1	2	4	8	12
0.0% failure	19ms	19ms	19ms	19ms	19ms
0.1% failure	19ms	19ms	19ms	19ms	20ms
1.0% failure	19ms	19ms	20ms	20ms	23ms
10% failure	23ms	34ms	59ms	168ms	247ms

In the first column we see that runtime increases with higher failure rates, but that increase is modest and to be expected. After all, exceptions are for "exceptional" situations, and thus 10% failure rate is already quite high. When we look at the last column with 12 threads the increase happens much earlier, though, already at 1% failure the execution time has grown significantly, and at 10% the overhead is unacceptable.

These numbers were measured on a Linux system using gcc 11.2, but we saw similar results with clang 13 and with the Microsoft C++ compiler on Windows. The root cause is that the unwinder grabs a global mutex to protect the unwinding tables from concurrent changes from shared libraries. This has disastrous performance implications on today's and upcoming machines. The Ryzen CPU shown above is a simple desktop CPU, when we do the same experiment on a dual socket AMD EPYC 7713 with 128 cores and 256 execution contexts we get the following numbers:

Threads	1	2	4	8	16	32	64	128
0.0% failure	24ms	26ms	26ms	30ms	29ms	29ms	29ms	31ms
0.1% failure	29ms	29ms	29ms	29ms	30ms	30ms	31ms	105ms
1.0% failure	29ms	30ms	31ms	34ms	58ms	123ms	280ms	1030ms
10% failure	36ms	49ms	129ms	306ms	731ms	1320ms	2703ms	6425ms

There, we start to get performance problems already at 0.1% failure rate, and the system becomes unusable at 1% failure rate or more. This makes it hard to justify using exceptions in C++, their performance is hard to predict and they degrade badly under high concurrency.

On the other hand, and in contrast to most of the alternatives discussed below, traditional C++ exceptions do have the advantage that they have (nearly) zero overhead compared to no error checking at all as long as no exception occurs. We can measure that with a code fragment that performs a very high number of function invocations and little extra work per call:

```
struct invalid_value {};

unsigned do_fib(unsigned n, unsigned max_depth) {
    if (!max_depth) throw invalid_value();
```

```

    if (n <= 2) return 1;
    return do_fib(n - 2, max_depth - 1) + do_fib(n - 1, max_depth - 1);
}

```

On the Ryzen we get as execution time for 10'000 invocations with $n = 15$ (and a certain probability of `max_depth` being 13, which triggers an exception):

Threads	1	2	4	8	12
0.0% failure	12ms	12ms	12ms	14ms	14ms
0.1% failure	14ms	14ms	14ms	14ms	15ms
1.0% failure	14ms	14ms	14ms	15ms	15ms
10% failure	18ms	20ms	27ms	64ms	101ms

When using C++ exceptions the results are similar to the `sqrt` scenario from above. We include them here because for the alternatives that we discuss below the `fib` scenario is the worst case, and significantly more expensive than the `sqrt` scenario. And again we have the problem that performance degrades with increasing concurrency.

§ 2. Root cause

Traditional C++ exceptions have two main problems:

1) the exceptions are allocated in dynamic memory because of inheritance and because of non-local constructs like `std::current_exception`. This prevents basic optimizations like transforming a `throw` into a `goto`, because other parts of the program should be able to see that dynamically allocated exception object. And it causes problems with throwing exceptions in out-of-memory situations.

2) exception unwinding is effectively single-threaded, because the table driven unwinder logic used by modern C++ compilers grabs a global mutex to protect the tables from concurrent changes. This has disastrous consequences for high core counts and makes exceptions nearly unusable on such machines.

The first problem seems unfixable without language changes, there are many constructs like `"throw;"` or `current_exception` that rely upon that mechanism. Note that these can occur in any part of the program, in particular in any function that is called by a catch block that is not inlined, thus we usually cannot simply elide the object construction. The second problem could potentially be fixed by a sophisticated implementation, but that would definitively be an ABI break and it would require careful coordination of all components involved, including shared libraries.

§ 3. Alternatives

Quite a few alternatives to traditional exceptions have been proposed, we will now look at some of them. All approaches solve the global mutex problem, thus multi-threaded performance is identical to single threaded performance and we only show single-threaded results. Source code to report full performance number is available at [\[ep\]](#). The main problem most of the alternatives have is that while they handle the sqrt scenario just fine, most of them have a significant performance overhead for the fib scenario. Which makes it difficult to simply replace traditional exceptions.

§ 3.1. `std::expected`

The `std::expected` proposal [\[P0323R11\]](#) introduces a variant type that either holds a value or an error object, which can be used to propagate the error state as a return value instead of throwing an exception. This solves the performance problem for sqrt, but it has a significant runtime overhead for fib:

failure rate	0.0%	0.1%	1.0%	10%
sqrt	18ms	18ms	18ms	16ms
fib	63ms	63ms	63ms	63ms

Single threaded the fib code using `std::expected` is more than four times slower than using traditional exceptions. Of course the overhead is less when the function itself is more expensive, as in the sqrt scenario. Nevertheless the overhead is so high that `std::expected` is not a good general purpose replacement for traditional exceptions.

§ 3.2. `boost::LEAF`

Instead of passing potentially complex error objects around, the catch-by-value proposal [\[P2232R0\]](#) suggests that it is much more efficient to catch objects by value instead of by reference. When catching by value, the throw location can identify the accepting catch, and then directly place the error object into stack memory provided by the try/catch block. The error itself can be propagated as a single bit. When using the `boost::LEAF` implementation of such a scheme we get the following performance numbers:

failure rate	0.0%	0.1%	1.0%	10%
sqrt	18ms	18ms	18ms	16ms
fib	23ms	22ms	22ms	22ms

This has much less overhead than `std::expected`, but it is still not for free. For `fib` we see a slowdown of approx. 60% compared to traditional exceptions, which is still problematic.

Note that LEAF profits significantly from using `-fno-exceptions` here. When enabling exceptions the `fib` case needs 29ms, even though not a single exception is thrown, which illustrates that exceptions are not truly zero overhead. They cause overhead by pessimizing other code.

§ 3.3. throwing values

The throwing values proposal [\[P0709R4\]](#) (also known as "Herbceptions") suggests that we do not allow for arbitrary exceptions to be thrown, but instead use a specific exception class which can be passed efficiently using two register values. The exception indicator itself is passed using a CPU flag when returning from a function. This is a clever idea that we unfortunately cannot implement in pure C++ due to lack of control over the CPU flags. We have thus tested two alternatives, one pure C++ approximation, where the non-exceptional result value must be at most pointer sized for optimal performance, and one hard-coded Herbception implementation using inline assembler. The performance numbers are:

failure rate	0.0%	0.1%	1.0%	10%
C++ emulation				
<code>sqrt</code>	18ms	18ms	18ms	16ms
<code>fib</code>	19ms	18ms	18ms	18ms
assembler				
<code>sqrt</code>	18ms	18ms	18ms	16ms
<code>fib</code>	13ms	13ms	13ms	13ms

This is close to being an acceptable substitute to traditional C++ exceptions. There is still some slowdown on the happy path, when no exception occurs, but that overhead is small, approx. 25% when using C++ and approx. 10% when using assembler, in a scenario where we do nearly nothing except calling other functions. It overtakes traditional exceptions if failure rates are higher. And it is dramatically better in multi-threaded applications.

§ 3.4. fixing traditional exceptions

Even though none of the leading C++ compilers does so, it is in fact possible to implement contention free exception unwinding. We did a prototype implementation where we changed the gcc exception logic to register all unwinding tables in a b-tree with optimistic lock coupling. This allows for fully parallel exception unwinding, the different threads can all unwind in parallel without any need for

atomic writes as long as there are no concurrent shared library operations. Shared library open/close triggers a full lock, but that should be rare. With such a data structure we can unwind in parallel, and we get a multi-threaded performance that is nearly identical to the single-threaded case, both on 12 and on 128 cores.

That sounds like an ideal solution, but in practice this is hard to introduce. It breaks the existing ABI, and all shared libraries would have to be compiled with the new model, as otherwise unwinding breaks. In a way the other alternative mechanisms break the ABI, too, but there the breakage is local to the code that uses the new mechanisms. Changing the traditional unwinding mechanism requires coordination across all code artifacts that are linked together. This would only happen if the C++ standard mandates that unwinding has to be contention free, and even then the introduction of the new ABI would be difficult.

A less radical change would be to change the global mutex into an rwlock, but unfortunately that is not easily possible either. Unwinding is not a pure library function but a back and forth between the unwinder and application/compiler code, and existing code relies upon the fact that it is protected by a global lock. In libgcc the callback from `dl_iterate_phdr` manipulates shared state, and switching to an rwlock leads to data races. Of course it would make sense to change that, but that would be an ABI break, too.

And fundamentally the current exception design is suboptimal for efficient implementations. For example we would like to be able to do the following transformation:

```
struct ex {};  
...  
int x;  
try {  
    if (y<0) throw ex{};  
    x=1;  
} catch (const ex&) {  
    foo();  
    x=2;  
}  
  
=>  
  
int x;  
if (y<0) { foo(); x=2; } else x=1;
```

But we cannot, as the function `foo()` might contain surprises like this:

```
void foo() {  
    if (random() < 10) some_global = std::current_exception();  
    if (random() < 100) throw;  
}
```

This forces exceptions to be globally available at all time and prevents more efficient implementations. And we saw these limitations in practice: Even with fully lock-free unwinding, we encountered some scalability issues with very high threads counts and high error rates (256 threads, 10% failure). These were far less severe than with current single-threaded unwinding, but nevertheless it is clear that the other parts of traditional exception handling do not scale either due to global state. Which is a strong argument for preferring an exception mechanism that uses only local state.

§ 4. Moving Forward

The current C++ exception mechanism has to change to stay relevant. Mainstream machines will soon have 256 cores and more, and current implementations cannot cope with that. The main question is which mitigation strategy should we use?

Throwing values [\[P0709R4\]](#) seems quite attractive, as it is one of the fastest approaches, is lock free, allows for transforming throw into goto, and does not require global coordination across all libraries. What is missing, however, is a way to integrate that mechanism into the existing language, in particular the standard library. The mechanism will be opt-in in the sense that we have to recompile the source code to get the throwing-values mechanism, but that is fine. The question is how can we get compatibility on the source level? Switching mechanisms based upon compiler flags seems dangerous with regards to the ODR, and switching from, e.g., `std::` to `std2::` would be a very invasive change. It is not clear yet what the best strategy would be. But something has to be done, as otherwise more and more people will be forced to use `-fno-exceptions` and switch to home grown solutions to avoid the performance problems on modern machines.

§ 5. Acknowledgments

- Thanks to Emil Dotchevski and Peter Dimov for their feedback

§ References

§ Informative References

[EP]

Thomas Neumann. *C++ exception performance experiments*. URL: <https://github.com/neumannt/exceptionperformance>

[P0323R11]

JF Bastien, Jonathan Wakely, Vicente Botet. *std::expected*. URL: <https://wg21.link/p0323r11>

[P0709R4]

Herb Sutter. *Zero-overhead deterministic exceptions: Throwing values*. URL: <https://wg21.link/p0709r4>

[P2232R0]

Emil Dotchevski. *Zero-Overhead Deterministic Exceptions: Catching Values*. URL: <https://wg21.link/p2232r0>